

Setting Goals

Overview

01 Sept 2026

- estimate the requirements to translate experimental notebooks and proof-of-concept models into scalable, maintainable production code
- identify metrics for complete machine learning pipelines from data ingestion through model deployment

Readings

Mitchell M, Wu S, Zaldivar A, et al. Model Cards for Model Reporting. In: Proceedings of the Conference on Fairness, Accountability, and Transparency. ACM; 2019:220-229.

<https://arxiv.org/pdf/1810.03993>

Brownlee, J. (2021, April 30). Tour of Evaluation Metrics for Imbalanced Classification.

MachineLearningMastery.com. <https://machinelearningmastery.com/tour-of-evaluation-metrics-for-imbalanced-classification/>

Model Cards for Model Reporting

Central Idea

Machine-learning models are increasingly used in high-impact areas such as:

- healthcare
- employment
- education
- law enforcement
- criminal justice

	However, released models often include little documentation about: • their intended applications • their performance limitations • the populations on which they were evaluated • the errors they are likely to make • the contexts in which they should not be used The authors propose model cards as a standardized form of model documentation. • a model card is a short document—typically one or two pages—that accompanies a trained model ○ it explains how the model was: ▪ developed ▪ evaluated ▪ intended to be used
Why Model Cards Are Needed	Overall performance metrics can conceal important differences between groups. • for example: ○ a classifier may achieve high aggregate accuracy while producing substantially higher error rates for: ▪ particular racial groups ▪ particular gender groups ▪ older or younger people ▪ people with particular skin types ▪ intersections of these categories

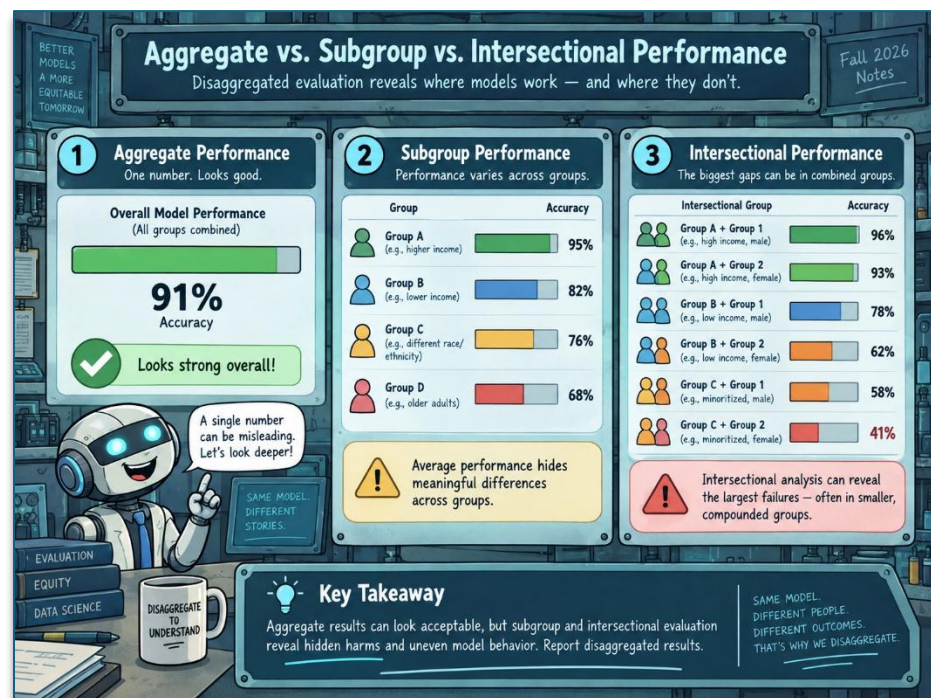
Many model failures have historically been discovered only after deployment, when affected users reported them.

- model cards encourage developers to **investigate** and **disclose** these differences before a model is adopted.

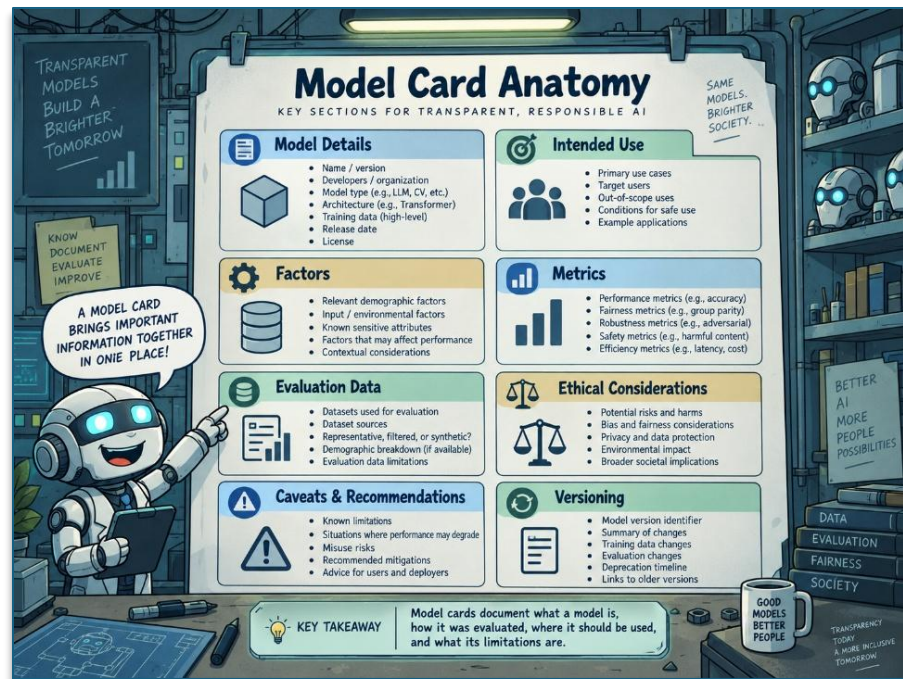
Disaggregated Evaluation

A central recommendation is to report model performance separately across relevant factors.

- these factors may include:
 - cultural groups
 - demographic groups
 - phenotypic characteristics
 - environmental conditions
 - input-capture technologies
 - intersections of multiple characteristics
 - unitary analysis



	Unitary analysis examines one factor at a time. • examples include: ○ performance by age ○ performance by gender ○ performance by geographic region ○ intersectional analysis Intersectional analysis examines combinations of factors. • examples include: ○ age and gender ○ race and gender ○ gender and skin type ○ age, disability, and geographic location A model may perform adequately for each broad category while failing for a particular intersection. • for example: ○ aggregate results for women and Black users may appear acceptable even when results for Black women are poor
Defining Sensitive Groups	Group-based evaluation introduces ethical and methodological problems. • categories such as race and gender: ○ are socially constructed ○ may not have objective ground-truth labels ○ can be incorrectly inferred from appearance ○ may not reflect how individuals identify themselves ○ can create privacy risks



ChatGPT 5.6 Sol 1

- where possible, the authors recommend using:
 - self-identified labels
 - labels explicitly described as perceived rather than self-identified
 - public identity information when evaluating public figures

Teams should collaborate with privacy, legal, policy, and domain experts before collecting or inferring sensitive attributes.

Proposed Model Card Structure

Model Details

This section identifies the model and its origin.

- it should include:
 - developer or responsible organization
 - model date
 - model version

	○ model type and architecture ○ training algorithms and important parameters ○ fairness constraints or mitigation methods ○ relevant papers or documentation ○ citation information ○ license ○ contact information for questions or feedback Version information is important because model behavior can change substantially between releases. • organizations are not expected to reveal protected intellectual property • they should still provide enough information for stakeholders to understand what the model represents
Intended Use	This section defines where and how the model should be used. • it should identify: ○ primary intended applications ○ intended users ○ intended operating environments ○ supported input types ○ uses that are explicitly out of scope Out-of-scope uses function like warnings on physical products. • examples might include: ○ not intended for emotion detection ○ not validated for medical diagnosis ○ not evaluated for text shorter than 100 tokens ○ designed only for black-and-white images

	○ not appropriate for autonomous decision-making A technically functional model is not necessarily appropriate for every related task.
Factors	Factors are conditions under which model performance may vary. Groups ● relevant population characteristics may include: ○ age ○ gender ○ race ○ disability ○ language ○ geographic location ○ Fitzpatrick skin type ● the relevant groups depend on the intended use and the populations most vulnerable to harm Instrumentation ● performance can vary according to the technology used to collect inputs ● examples include: ○ camera type ○ lens type ○ image stabilization ○ microphone type ○ recording software ○ sensor resolution ○ aperture, shutter speed, or ISO

	○ environment Environmental conditions may also affect performance. • examples include: ○ lighting ○ humidity ○ background noise ○ weather ○ network quality ○ physical location The model card should distinguish between: • relevant factors: ○ conditions likely to affect performance • evaluation factors: ○ conditions actually included in testing • any gap between these categories should be explained
Metrics	Metrics should reflect the model's intended use and the consequences of different errors. • for classification systems, relevant measures may include: ○ false-positive rate ○ false-negative rate ○ false-discovery rate ○ false-omission rate ○ precision ○ recall ○ decision thresholds

	The importance of each error depends on the application and stakeholder. • for example, in surveillance: ○ operators may prioritize a low false-negative rate ○ people being surveilled may prioritize a low false-positive rate A model card should report multiple relevant metrics and explain which were prioritized.
Thresholds and Uncertainty	When a model converts scores into decisions, the model card should document: • the decision threshold • why the threshold was selected • how performance changes at other thresholds • statistical uncertainty in the reported results Uncertainty measures may include: • standard deviation • variance • confidence intervals • results across repeated runs • cross-validation results Confidence intervals are especially important for small subgroups. • an apparently large performance difference may be unreliable when few evaluation examples are available

**Evaluation
Data**

The model card should document the data used to test the model.

- this should include:
 - evaluation datasets
 - reasons for selecting each dataset
 - dataset composition
 - preprocessing procedures
 - filtering or exclusion rules
 - known limitations

Evaluation data should represent:

- normal operating conditions
- anticipated use cases
- difficult or unusual cases
- populations that may be especially vulnerable to errors

Public evaluation datasets are valuable because they allow independent verification and comparison.

- synthetic data may be useful when real data does not adequately represent important cases
- however, synthetic data can test only the patterns included in its design

Training Data	Ideally, the model card should provide training-data information comparable to the evaluation-data documentation. • this may not always be possible because the data could be: ○ proprietary ○ confidential ○ protected by nondisclosure agreements ○ subject to privacy restrictions When full disclosure is impossible, developers should provide basic information about: • data sources • population distributions • relevant group representation • known gaps • potential biases encoded in the data
Quantitative Analysis	Evaluation results should be separated by the factors defined earlier. • the card should include: ○ unitary results for individual factors ○ intersectional results for combinations of factors ○ confidence intervals ○ error bars or other measures of variation Group parity in some metrics corresponds to formal definitions of fairness. • for example: • equal false-negative rates relate to equality of opportunity • equal false-positive and false-negative rates relate to equalized odds

	However, no single fairness metric is appropriate for every application. • metric selection must reflect: ○ deployment context ○ distribution of potential harms
Ethical Considerations	The model card should explain the ethical issues considered during development. • questions may include: • could the model cause harm? • which people or groups could be affected? • could harms differ across groups? • could the model be used in especially sensitive settings? • were affected communities consulted? • was the system reviewed by an external board? • what mitigation measures were applied? • which risks remain unknown? Ethical analysis may not produce a complete solution. • its purpose is to make risks, assumptions, and unresolved questions visible
Caveats and Recommendations	This section documents remaining limitations. • it should identify: ○ groups missing from the evaluation data ○ factors that were not tested ○ known performance weaknesses ○ additional testing requirements ○ conditions under which the model should not be used

	○ recommendations for fine-tuning or further evaluation ○ characteristics of an improved evaluation dataset
Who Benefits from Model Cards?	ML practitioners ● evaluate suitability for an intended application ● compare candidate models ● monitor performance across versions ● identify areas requiring additional training Software developers ● understand model constraints ● design appropriate interfaces and fallbacks ● select decision thresholds ● avoid unsupported applications Organizations ● make informed adoption decisions ● compare vendors and models ● evaluate operational and ethical risks Policymakers and regulators ● identify questions that should be asked before deployment ● understand possible failure patterns ● assess whether a model is appropriate for a high-impact setting Affected individuals. ● understand how a model may influence them ● identify documented limitations

	• use disclosed information to challenge or seek remedies for harmful decisions
Limitations of Model Cards	Model cards do not guarantee responsible behavior. • their usefulness depends on: ○ the honesty of their creators ○ the quality of the evaluation ○ the relevance of the selected groups and metrics ○ whether important limitations are disclosed ○ whether documentation is updated when the model changes A developer could unintentionally or deliberately present misleading results. • model cards should therefore complement: ○ independent algorithmic audits ○ adversarial testing ○ qualitative evaluation ○ community consultation ○ inclusive feedback mechanisms ○ regulatory oversight They are one transparency mechanism, but not a substitute for governance or accountability.

Evaluation Metrics for Imbalanced Classification

Central Idea

A classification model is only as useful as the metric used to evaluate it.

- this is especially important for imbalanced classification, where one class contains substantially fewer observations than another
 - examples include:
 - fraud detection
 - disease screening
 - equipment-failure prediction
 - cyberattack detection
 - customer-churn prediction
 - rare-event forecasting

In these problems, the minority class is often the class of greatest practical importance.

- a poorly chosen metric can:
 - make an ineffective model appear successful
 - hide poor minority-class performance
 - select the wrong model
 - encourage an unsuitable decision threshold
 - produce misleading conclusions about deployment performance

Why Accuracy Can Be Misleading	Accuracy measures the proportion of all predictions that are correct: $Accuracy = \frac{Correct\ predictions}{Total\ predictions}$ Suppose a dataset contains: • 990 negative cases • 10 positive cases A model that predicts every case as negative achieves: $Accuracy = \frac{990}{1000} = 0.99$ • the model has 99% accuracy, but it detects none of the positive cases • therefore, accuracy can be dangerously misleading when: ○ one class strongly dominates the dataset ○ the minority class is operationally important ○ different error types have different consequences Model performance must instead be evaluated using metrics aligned with the application's objectives and error costs.
Async	
Setting Goals and Defining Success Metrics in ML products	
Overview	• machine learning products require success criteria that extend beyond model performance

	• effective goals connect technical metrics to business outcomes and operational constraints across the full ML system lifecycle
From Prototype To Product	ML product development should consider the full pipeline: • problem definition • data preparation • model building • deployment • monitoring Cloud deployments introduce additional considerations: • scalability • cost • compliance • integration with other services Success is multidimensional and includes: • technical performance • business outcomes • operational stability
The Accuracy Trap	A common ML project failure is optimizing model accuracy without determining whether higher accuracy supports the actual objective. • clear goals help ensure that: ○ improvements in model metrics translate into useful outcomes ○ design choices reflect the intended objective

- experiments are evaluated against appropriate benchmarks
- teams focus on metrics that matter to project success
- iteration remains measurable and purposeful

Goal Alignment

ML systems can be viewed as having three connected layers of goals.

Business Goals:

- business goals describe the high-level outcomes the organization wants to achieve
 - examples:
 - increase customer retention
 - reduce costs
 - reduce financial losses
 - improve user satisfaction
 - increase engagement
 - improve safety



ChatGPT 5.6 Sol

	ML Goals: • ML goals translate business objectives into measurable model objectives ○ examples: ▪ recall for a churn model ▪ precision for a fraud detector ▪ classification accuracy ▪ prediction quality associated with user activity • ML metrics often serve as proxies for outcomes that cannot be measured directly System Goals: • system goals define the operational conditions required for the ML product to function effectively ○ examples: ▪ response time ▪ throughput ▪ uptime ▪ cost per prediction All three layers should remain aligned. • a technically strong model provides little value if it does not: ○ improve the relevant business outcome ○ cannot operate within the system's constraints
Fraud Detection Example	Business goal: • reduce financial losses caused by fraud

	ML goal: • maximize recall • detect at least 90% of fraudulent transactions System goal: • evaluate each transaction in under 200 milliseconds The goals are aligned when higher recall reduces fraud losses while the system processes fraud checks quickly enough to avoid disrupting customer transactions.
Using Proxy Metrics	Business objectives are often qualitative and therefore require measurable proxies. • engagement proxies: ○ average session length ○ daily active users • safety proxy: ○ incident rate per thousand hours These business proxies can then be connected to appropriate ML metrics, such as classification performance for incident detection or predictions associated with user activity.
Monitoring Success	Business proxies and ML metrics should be monitored together.

	• monitoring both levels helps determine whether improvements in model performance are producing meaningful improvements in the real-world objective. Core Principle: • model metrics are useful: ○ when they support the broader product objective • successful ML products align: ○ business goals ○ ML goals ○ system goals • so that technical improvements translate into: ○ measurable value under realistic operating constraints
Machine Learning Evaluation Metrics	
Classification Metrics	Model evaluation metrics should be selected based on the type of: • machine learning task • real-world consequences of different errors • classification tasks include: ○ spam detection ○ medical diagnosis ○ fraud detection • accuracy ○ fraction of all predictions that are correct • precision

	○ fraction of predicted positive cases that are actually positive ○ high precision means fewer false positives or false alarms ● recall ○ fraction of actual positive cases correctly identified ○ high recall means fewer false negatives or missed cases ● F1 score ○ harmonic mean of precision and recall ○ provides a single measure that balances precision and recall ● ROC-AUC ○ area under the Receiver Operating Characteristic curve ○ summarizes classifier performance across multiple decision thresholds ● applications can prioritize: ○ minimizing false negatives ○ minimizing false positives Metric selection depends on which errors are most costly.
Regression Metrics	Regression models predict continuous values such as prices, demand, or quantities. Root Mean Squared Error (RMSE) ● square root of the average squared prediction error

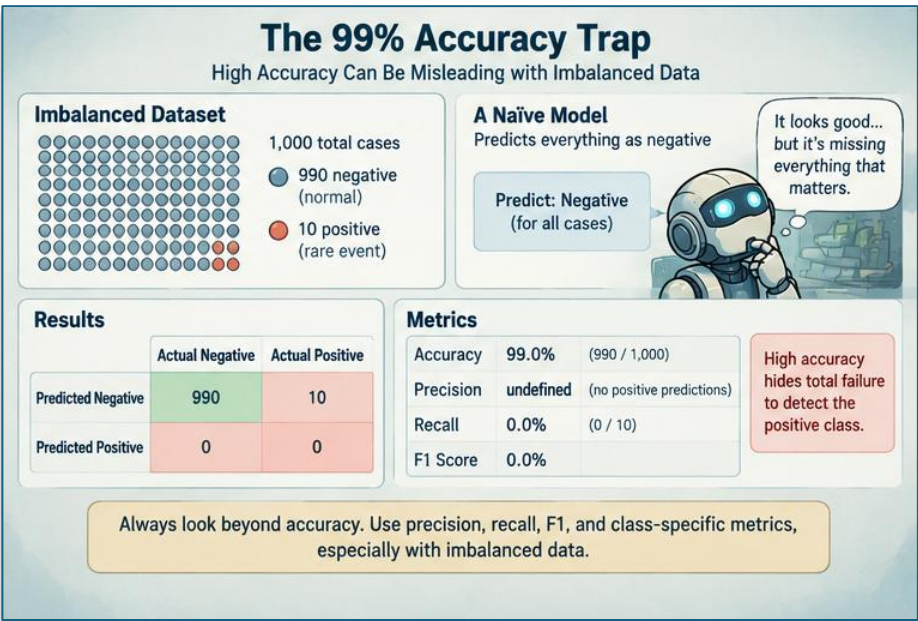
	• penalizes large errors more strongly than small errors • useful when large prediction errors are especially costly Mean Absolute Error (MAE) • average absolute difference between predicted and actual values • treats errors linearly • provides an intuitive measure of average error magnitude R-squared • proportion of variance in the target variable explained by the model • $R_{squared} = 1$ ○ indicates a perfect fit • $R_{squared}$ near 0 ○ indicates performance close to predicting the target mean RMSE and MAE are expressed in the same units as the target variable.
Ranking Metrics	Recommendation systems and search engines require metrics that evaluate the order in which results are presented. Normalized Discounted Cumulative Gain (NDCG) • gives greater weight to relevant results appearing near the top of the ranking Mean Average Precision (MAP) • averages precision values at relevant result positions across queries

	Mean Reciprocal Rank (MRR) • focuses on the position of the first relevant result Ranking metrics are especially useful when users need to encounter useful results quickly.
Unsupervised Learning Metrics	Unsupervised learning methods such as clustering also require evaluation. Silhouette score • ranges from -1 to 1 • combines cluster cohesion and cluster separation • higher values: ○ indicate better-defined clusters • negative values: ○ may indicate that observations are assigned to inappropriate clusters Cohesion • how close observations are within the same cluster Separation • how far different clusters are from one another Davies-Bouldin index • evaluates similarity between: ○ clusters using within-cluster dispersion

	○ between-cluster separation • lower values: ○ indicate better clustering Calinski-Harabasz index • ratio of between-cluster variance to within-cluster variance • higher values: ○ indicate clusters that are more distinct and cohesive
Metrics And Error Costs	Choosing an evaluation metric is not only a technical decision; metrics should represent the practical cost of incorrect predictions. Health care diagnosis: • false negatives: ○ may be especially costly because a disease case could be missed • recall or sensitivity: ○ may therefore receive greater emphasis Spam filtering: • false positives: ○ may be especially costly because • legitimate messages: ○ could be incorrectly removed • precision: ○ may therefore receive greater emphasis

	The appropriate metric depends on the application's objectives and the relative cost of each type of error.
Avoiding Single-Metric Fixation	Model evaluation should not rely on a single metric without considering its relationship to business or application objectives. • metrics should: ○ align with relevant business KPIs ○ reflect the consequences of false positives and false negatives ○ support comparison between competing models ○ account for operational requirements ○ be interpreted alongside model complexity
Fraud Detection Trade-Off	Increasing recall causes the classifier to identify a larger proportion of fraudulent transactions. • high recall: ○ fewer fraudulent transactions are missed ○ more legitimate transactions may be incorrectly flagged ○ precision may decrease ○ false positives may increase Increasing precision causes the classifier to flag transactions only when it has greater confidence that fraud is present. • high precision: ○ fewer legitimate transactions are incorrectly flagged ○ more fraudulent transactions may be missed ○ recall may decrease ○ false negatives may increase

	• false positive costs: ○ customer inconvenience ○ blocked legitimate purchases ○ reduced user experience ○ potential lost revenue • false negative costs: ○ undetected fraudulent transactions ○ direct financial loss
Decision Thresholds	Classification models often produce scores or probabilities that must be converted into class predictions using a decision threshold. Changing the threshold changes the balance between: • precision • recall • false positive rate • false negative rate



ChatGPT 5.6 Sol

The appropriate threshold should reflect the real-world costs associated with each error type.

- cost matrices and performance curves:
 - can help identify an acceptable threshold

Stakeholders such as finance teams and customer experience teams provide domain knowledge needed to determine the appropriate balance.

ROC Curves

ROC: Receiver Operating Characteristic.

- evaluates binary classifier performance across possible decision thresholds

The curve compares:

- true positive rate
- false positive rate

	A curve closer to the upper-left corner indicates that the model can achieve high recall while maintaining a low false positive rate.
AUC	AUC: Area Under the ROC Curve. • provides a single-number summary of ranking performance across thresholds. • $AUC = 1.0$ ○ perfect discrimination between positive and negative examples • $AUC = 0.5$ ○ performance equivalent to random ranking AUC can also be interpreted as: • the probability that a randomly selected positive example receives a higher score than a randomly selected negative example AUC should not be used in isolation. • model evaluation should also examine: ○ threshold-specific behavior ○ application costs ○ operational constraints ○ model complexity
Core Principle	Evaluation metrics should be selected according to: • the machine learning task

	- the costs of different errors - the objectives of the broader system Strong model evaluation requires balancing multiple metrics rather than optimizing a single value without considering its real-world consequences.
Rare Events and Adjusting Thresholds	
Threshold Tuning, Operational Metrics, And Business Alignment	Model evaluation in production requires more than choosing a standard accuracy metric. - rare events all affect whether a model is actually successful - threshold selection - operational constraints - human feedback - business outcomes
Rare Events And Precision-Recall Curves	Rare-event problems include: - fraud detection - disease detection - network interruptions ROC curves can be less informative when the positive class is rare. - performance may appear strong even when the model performs poorly on the minority class Precision-recall curves. - often more informative for rare-event problems - plotted against recall as the classification threshold changes

	○ precision-recall curves emphasize performance on the positive class • when positive cases are rare ○ random or trivial models typically have very low precision ○ improvements over the baseline are therefore easier to observe The precision-recall curve can reveal where increasing recall begins to cause a substantial decline in precision. • this balance can be used to select a threshold that reflects application requirements ○ example objective: ▪ maintain at least a specified level of precision while maximizing recall
Threshold Tuning	Binary classifiers commonly use 0.5 as a default decision threshold. • this value should not automatically be treated as optimal The threshold should be adjusted according to: • business objectives • error costs • precision requirements • recall requirements When false negatives are especially costly: • lower the decision threshold • increase recall

	• capture more positive cases • accept more false positives When false positives are especially costly: • raise the decision threshold • increase precision • reduce unnecessary alerts • accept more false negatives Threshold tuning should be performed using: • validation data • cross-validation Threshold tuning should not be performed directly on the training data because this can overfit the threshold to the training sample.
Cost-Based Threshold Selection	False positives and false negatives can sometimes be assigned explicit financial costs. • expected cost can be computed across possible thresholds • the selected threshold can then minimize the total expected cost This approach directly connects classifier behavior to the economic consequences of prediction errors.
Operational Metrics	Production ML systems must also be evaluated using operational metrics. • important operational metrics include: ○ latency

	○ throughput ○ cost per prediction Latency. • measures how long a prediction takes ○ example: ▪ required <i>response time</i> = 100 milliseconds ▪ actual <i>response time</i> = 500 milliseconds • a model that exceeds the required latency may be unsuitable for deployment regardless of its predictive accuracy Throughput. • measures how many predictions the system can process within a given period ○ example: ▪ 1,000 predictions per second • the deployed system must support the expected volume of requests under realistic operating conditions Cost Per Prediction. • measures the computational cost associated with generating each prediction A highly accurate model may still be impractical if the required cloud or GPU resources make inference too expensive at scale.
Operational Trade-Offs	Operational metrics often trade off against model quality and against one another.

	• example: ○ a larger neural network may improve predictive accuracy ○ inference may become slower ○ computational cost may increase Success criteria should therefore include both predictive and operational requirements.
Multi-Metric Monitoring	Production teams often monitor several metrics together. • a multi-metric dashboard may include: ○ accuracy ○ precision ○ recall ○ latency ○ throughput ○ cost Monitoring these metrics together helps prevent optimization of one dimension at the expense of the overall system.
Qualitative Success Metrics	Not every important property of an ML system can be represented adequately by an automated numerical metric. • qualitative factors may include: ○ user satisfaction ○ perceived relevance ○ ethical considerations ○ domain expert judgment

Human Evaluation	Human feedback can supplement automated metrics. • possible feedback sources include: ○ structured user surveys ○ user ratings ○ domain expert review ○ internal moderators ○ sampled real-world interactions For some applications, human evaluation may provide the most meaningful measure of output quality.
Human-In-The-Loop Evaluation	Human-in-the-loop systems combine automated model evaluation with human judgment. • human feedback ○ can be collected on a subset of interactions • qualitative feedback ○ can be compared with automated metrics over time • large disagreements between the two may indicate an important problem ○ example: ▪ model accuracy is high ▪ user satisfaction is low This divergence suggests that the automated metric may not fully capture the quality users actually experience.

Closing The Loop

Model improvements should ultimately produce measurable improvements in business outcomes.

- a key evaluation question is:
 - does improvement in the model metric produce improvement in the relevant business KPI?
- example:
 - churn model recall increases from 80% to 85%
 - customer churn does not decline

Possible explanations include:

- additional predictions may include more false positives
- the model may identify customers who are unlikely to respond to intervention
- the organization may lack an effective intervention strategy

Improved model performance does not automatically create business value.

- business KPIs should therefore be monitored alongside model metrics

Production ML evaluation requires balancing:

- predictive performance
- threshold behavior
- operational constraints
- qualitative feedback
- business outcomes.

	A model improvement is meaningful only when it produces useful real-world results within acceptable technical and operational limits.
Data Validation	
Data Validation, Drift, And Pipeline Monitoring	Reliable ML metrics depend on reliable data. - production systems therefore require continuous validation of: - incoming data - monitoring for drift - pipeline health checks - alerting - experiment tracking
Data Validation At Scale	Data validation checks whether incoming data matches the assumptions used during model development. - schema validation: - confirm required fields are present - verify expected data types - detect upstream changes that could corrupt model inputs - range and distribution checks: - define plausible minimum and maximum values - compare incoming distributions with expected ranges - flag or filter values outside acceptable limits - examples: - age must fall between 0 and 100 - sensor readings should not be negative

Missing Data	Missing values require an explicit handling strategy. • possible approaches: ○ drop affected rows ○ use default placeholders ○ apply data imputation techniques At minimum, the proportion of missing values should be monitored for every feature.
Data Drift	Data drift occurs when the distribution of model inputs changes after deployment. • examples: ○ average user age changes ○ new slang appears in a text corpus ○ sensor values shift over time Data drift. • does not necessarily mean that the relationship between features and outcomes has changed • it can cause a model to encounter data that differs substantially from its training data • drift detection may compare: ○ current production data ○ training data ○ historical production data

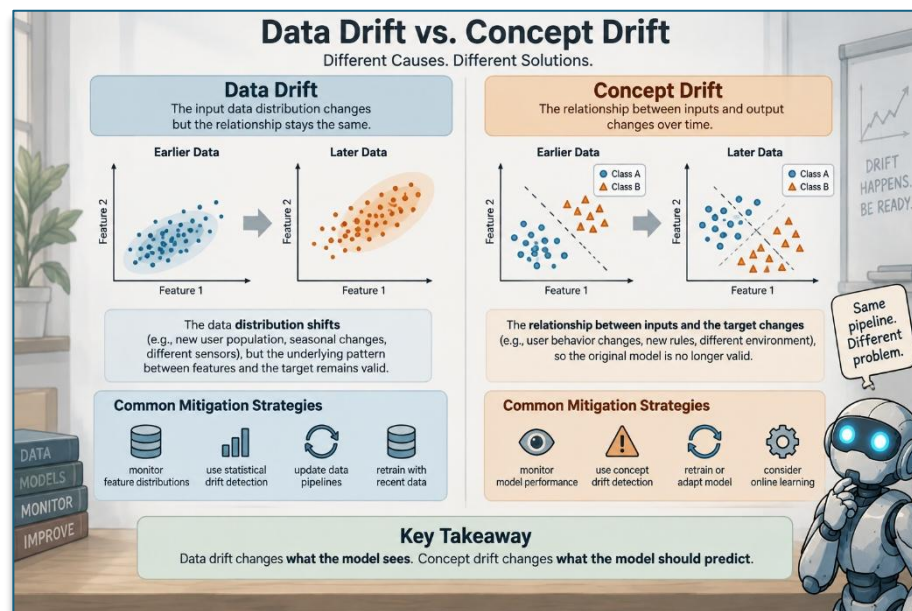
Population Stability Index

Population Stability Index, or PSI, can be used to compare the distribution of current input data with the training distribution.

- if drift exceeds an established threshold:
 - generate an alert
 - investigate the affected features
 - consider retraining the model

Concept Drift

Concept drift occurs when the relationship between model inputs and the target outcome changes.



ChatGPT 5.6 Sol

- example:
 - behavior that previously indicated customer churn
 - no longer predicts churn
 - because market conditions have changed
- possible causes:
 - new competitors

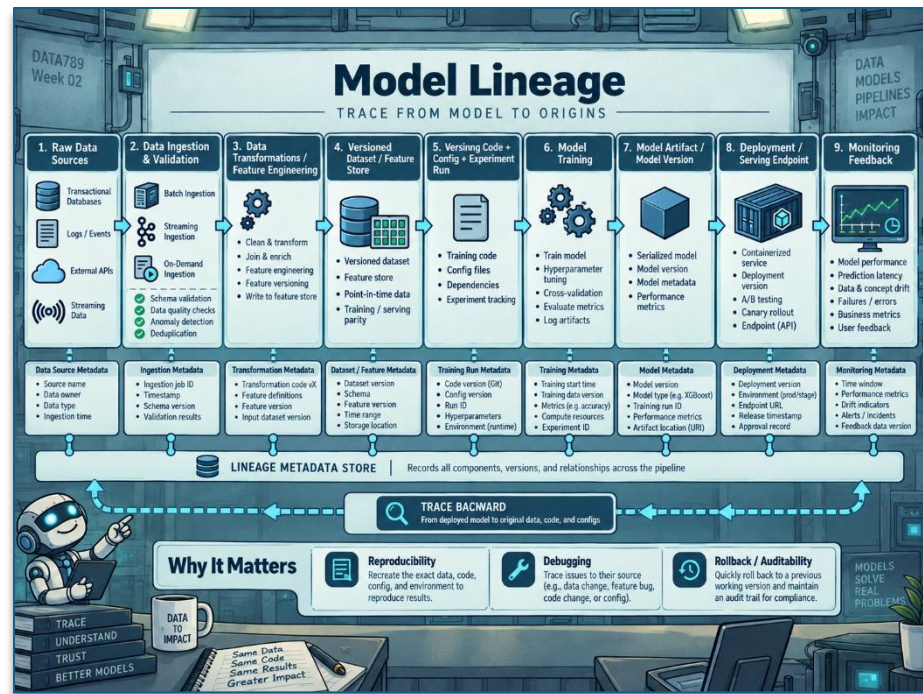
	○ competitors leaving the market ○ changes in customer behavior ○ changes in external conditions Concept drift can silently reduce model performance even when the incoming data still appears valid.
Monitoring	Concept drift can be detected by monitoring model performance over time. ● possible indicators: ○ accuracy declines ○ recall declines ○ precision declines ○ other production metrics fall below an established baseline Ground-truth labels. ● may arrive after predictions are made ● can be calculated over rolling windows once outcomes become available
A/B Testing	Can also help identify concept drift. ● example: ○ deploy a new model to a subset of traffic ○ compare its performance with the existing model Significantly better performance from the new model may indicate that the existing model has degraded.

ML Pipeline Health	A strong model is not useful if the surrounding pipeline fails to deliver reliable predictions. • production monitoring should therefore include the entire ML pipeline • job success and failure rates: ○ monitor scheduled pipelines ○ detect failed ETL processes ○ identify incomplete data updates • example: ○ a nightly ETL job fails 10% of the time ○ approximately 10% of expected updates may be missing
Component Runtime	The duration of each pipeline component should be monitored. • example: ○ stream preprocessing suddenly requires twice as much time An increase in processing time can create bottlenecks and threaten real-time prediction requirements.
Resource Utilization	Important resource metrics include: • CPU utilization • memory utilization • GPU utilization High utilization may indicate: • insufficient capacity • need for scaling • inefficient code

	Very low utilization may indicate: • overprovisioning • unnecessary infrastructure cost • underutilized test environments
Alerts And Automated Responses	Monitoring is useful only when detected problems trigger an appropriate response. • alert conditions may include: ○ precision falls below 0.8 ○ latency exceeds an operational threshold ○ daily cost exceeds a defined limit ○ pipeline jobs fail ○ resource utilization exceeds capacity Alerts can trigger: • human notifications • automated remediation • service scaling • service restarts • fallback model activation Known and predictable failures should be automated when possible. • unknown failures or unsuccessful automated recovery should be escalated to humans Automated remediation is an important MLOps capability because it reduces downtime and helps maintain consistent service quality.

Experiment Tracking	Model development may involve many training runs for: • hyperparameter tuning • feature engineering • model comparison • architecture changes Experiment tracking records the configuration and results of each run. • commonly logged metrics include: ○ accuracy ○ loss ○ precision ○ recall Runs can then be compared visually or programmatically.
Additional Experiment Data	Useful experiment metadata may include: • confusion matrices • feature importance • training time • hyperparameters • model configuration • software versions Tracking more than the final evaluation metric makes model comparison, debugging, and reproducibility easier.

Data And Model Versioning	Production ML systems should maintain versions of: • datasets • models • code • training runs Versioning makes it possible to trace exactly how a deployed model was produced. • example lineage: ○ model version 5 ○ training run 42 ○ dataset version 3 ○ code version ABC123
Model Lineage	Model lineage creates a traceable relationship between: • source data • training configuration • code • experiment • model artifact • deployment This traceability supports: • reproducibility • debugging • governance • auditing



ChatGPT 5.6 Sol

Months after deployment:

- the team should still be able to determine how a specific model was trained

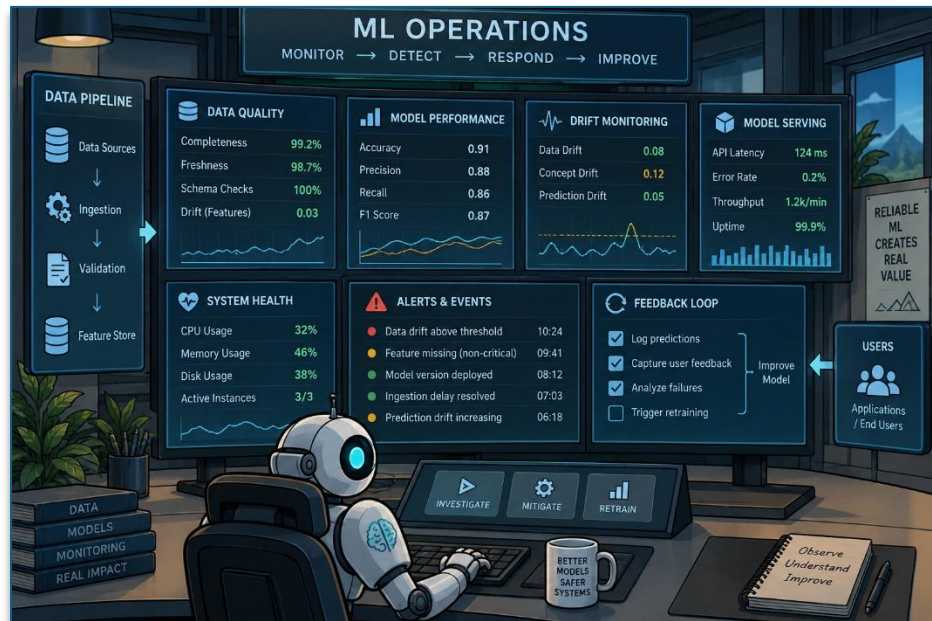
If a new model performs worse than a previous version, lineage also makes it easier to identify what changed.

Unified Monitoring

Technical and business metrics should be integrated into a shared monitoring environment.

- a unified dashboard may include:
 - model performance
 - data quality
 - drift metrics
 - pipeline health
 - latency
 - resource utilization

- business KPIs



ChatGPT 5.6 Sol

Different stakeholders can examine the same underlying system from different perspectives.

- executives may examine:
 - sales
 - engagement
 - regional KPIs
- engineers may examine:
 - API latency
 - resource utilization
 - pipeline failures
- data scientists may examine:
 - feature distributions
 - data drift
 - model performance

	○ prediction behavior
Shared Source Of Truth	A unified monitoring system allows business and technical teams to work from the same underlying information. ● benefits include: ○ faster diagnostics ○ earlier detection of problems ○ better collaboration ○ reduced dependence on delayed reports ○ consistent interpretation of system health
Core Principle	Production ML systems require continuous validation of: ● data ● models ● pipelines ● business outcomes Effective monitoring combines: ● data quality checks ● drift detection ● operational health ● automated responses ● experiment tracking ● model lineage ● shared dashboards Problems can be: ● detected

- diagnosed
- addressed quickly